Анализ данных с использованием языка Python

Пояснения к коду. Семинар 13

Две выборки: t-тест, критерий Манна–Уитни, хи-квадрат

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-13-two-samples.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 8–12, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/statistics.md`, раздел «Две и больше выборок».

1 Две группы: t-тест Уэлча

Ячейка 1. Подключение и стаж по исходу ДТП

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  import pandas as pd
4  import seaborn as sns
5  from scipy import stats
6
7  sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
8  dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
   ↪ parse_dates=['datetime'])  # ДТП Брянской области
9  people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # ↩
   ↪ участники ДТП
10 people['experience_clean'] = people['experience'].replace([95, 96, 99], ↩
   ↪ np.nan)  # коды стажа пропусками
11 drivers = people[people['role'] == 'Водитель'].merge(dtp, ↩
   ↪ left_on='dtp_id', right_index=True)  # водители с их ДТП
12 drivers['vehicle_age'] = drivers['datetime'].dt.year - ↩
   ↪ drivers['vehicle_year']  # возраст машины в годах
13 drivers['vehicle_age'] = ↩
   ↪ drivers['vehicle_age'].where(drivers['vehicle_age'] <= 100)  # ↩
   ↪ невозможный возраст в пропуск
14 drivers.groupby('health')['experience_clean'].agg(['count', 'mean', ↩
   ↪ 'median']).round(2)  # стаж по исходу
```

– Строки 1–11 — подключение и водители со своими ДТП, как на семинаре 12; statsmodels сегодня не нужен.

– `drivers.groupby('health')['experience_clean'].agg(['count', 'mean', 'median'])` — по каждому исходу для водителя, столбец `health`, три статистики стажа: число значений, среднее и медиана.

Ячейка 2. Две группы и ящики трёх исходов

```
dead = drivers.loc[drivers['health'] == 'Погиб', ↩
↪ 'experience_clean'].dropna()  # стаж погибших водителей
injured = drivers.loc[drivers['health'] == 'Ранен', ↩
↪ 'experience_clean'].dropna()  # стаж раненых водителей
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=drivers, x='health', y='experience_clean', ↩
↪ order=['Погиб', 'Ранен', 'Не пострадал'], ax=ax)  # ящики трёх исходов
ax.set_title('Стаж водителей по исходу ДТП')
ax.set_xlabel('Исход для водителя')
ax.set_ylabel('Стаж, лет');
```

– Строки 1–2 — стаж погибших и стаж раненых водителей без пропусков: две группы для сравнения.

– `sns.boxplot(..., x='health', y='experience_clean', order=[...])` — ящики стажа по исходам; `order` — порядок ящиков.

Ячейка 3. t-тест Уэлча

```
welch = stats.ttest_ind(dead, injured, equal_var=False)  # t-тест Уэлча
welch
```

– `stats.ttest_ind(dead, injured, equal_var=False)` — t-тест двух независимых выборок. `equal_var=False` — вариант Уэлча: дисперсии групп не предполагаются равными. Без этого параметра scipy считает тест Стьюдента.

– Результат — объект `TtestResult` с полями `statistic`, `pvalue` и `df`; у Уэлча число степеней свободы дробное.

Ячейка 4. Коэффициент d Коэна

```
pooled = np.sqrt((dead.var() + injured.var()) / 2)  # объединённое отклонение
d = (dead.mean() - injured.mean()) / pooled  # d Коэна
print(round(dead.mean() - injured.mean(), 2), round(pooled, 2), round(d, 2))
```

– `dead.var()` — дисперсия, квадрат стандартного отклонения. `np.sqrt((... + ...) / 2)` — корень из среднего двух дисперсий: объединённое отклонение двух групп.

– Строка 2 — d Коэна: разница средних, делённая на объединённое отклонение. Знак показывает, какая группа больше: плюс — первая.

Ячейка 5. Тест Стьюдента без equal_var: ловушка без ошибки

```
violators = drivers.loc[drivers['violation'] == 'Непредоставление преимущества в движении пешеходу', 'experience_clean'].dropna()  # не уступили пешеходу
lawful = drivers.loc[drivers['violation'] == 'Нет нарушений', 'experience_clean'].dropna()  # водители без нарушений
student = stats.ttest_ind(violators, lawful)  # по умолчанию - Стьюдент
print(len(violators), len(lawful), round(violators.std(), 1), round(lawful.std(), 1))
print(round(student.pvalue, 3), round(stats.ttest_ind(violators, lawful, equal_var=False).pvalue, 3))
```

– Строки 1–2 — стаж водителей с одним нарушением в столбце `violation` и водителей без нарушений. Значение пишется точно как в данных.

– `stats.ttest_ind(violators, lawful)` — без `equal_var` это тест Стьюдента: разброс групп считается одинаковым.

– Строка 4 — размеры групп и стандартные отклонения: у маленькой группы разброс больше.

– Строка 5 — p Стьюдента рядом с p Уэлча: один отвергает, другой нет.

2 Непараметрический двойник: Манна–Уитни

Ячейка 6. Критерий Манна–Уитни и доля пар

```
mw = stats.mannwhitneyu(dead, injured)  # критерий Манна-Уитни
n_pairs = len(dead) * len(injured)  # все пары «погибший, раненый»
ties = (dead.value_counts() * injured.value_counts()).sum()  # пары с равным стажем
print(mw.statistic, mw.pvalue, round(mw.statistic / n_pairs, 3))  # U, p и доля пар
print(round((mw.statistic - ties / 2) / n_pairs, 3), round(ties / n_pairs, 3))  # опытнее и поровну
```

– `stats.mannwhitneyu(dead, injured)` — критерий Манна–Уитни. `statistic` — U первой группы: в скольких парах «погибший, раненый» стаж погибшего больше; равный стаж — за половину пары.

– `len(dead) * len(injured)` — число всех пар.

– `dead.value_counts() * injured.value_counts()` — для каждого значения стажа: сколько погибших с таким стажем, умноженное на число раненых с тем же стажем. Series перемножаются по совпадающим меткам индекса; стаж, который есть только в одной группе, даёт пропуск, и `sum()` его пропускает. Сумма — пары с равным стажем.

– `mw.statistic / n_pairs` — доля пар, где погибший опытнее, вместе с половиной ничьих.

– Строка 5 — доля пар, где погибший строго опытнее, и доля ничьих.

Ячейка 7. Возраст машин у женщин и мужчин

```
1 women = drivers.loc[drivers['gender'] == 'Женский', ↩
↪ 'vehicle_age'].dropna()  # возраст машин женщин
2 men = drivers.loc[drivers['gender'] == 'Мужской', 'vehicle_age'].dropna()  ↩
↪ # возраст машин мужчин
3 age_test = stats.ttest_ind(women, men, equal_var=False)  # t-тест Уэлча
4 d_age = (women.mean() - men.mean()) / np.sqrt((women.var() + men.var()) / ↩
↪ 2)  # d Коэна
5 print(len(women), len(men), round(women.mean(), 2), round(men.mean(), 2), ↩
↪ women.median(), men.median())
6 print(age_test.pvalue, stats.mannwhitneyu(women, men).pvalue, round(d_age, ↩
↪ 2))
```

– Строки 1–2 — возраст машин водителей-женщин и водителей-мужчин без пропусков.

– Строки 3–4 — t-тест Уэлча и коэффициент d Коэна, как в ячейках 3 и 4, только формула d записана одной строкой.

– Строка 6 — p обоих тестов и d рядом: значимость и величина разницы.

3 Парные данные

Ячейка 8. Две группировки по годам

```
1 year = dtp['datetime'].dt.year  # год каждого ДТП
2 first = dtp[year == 2015].groupby('district').size()  # ДТП по районам, 2015
3 first = first[first >= 20]  # районы от 20 ДТП
4 last = dtp[year == 2025].groupby('district').size()  # ДТП по районам, 2025
5 print(len(first), len(last))
```

– `year = dtp['datetime'].dt.year` — год каждого ДТП, Series той же длины, что таблица.

– `dtp[year == 2015].groupby('district').size()` — число ДТП по районам в 2015 году; строки отобраны маской по Series `year`.

– `first[first >= 20]` — районы от 20 ДТП в 2015 году. Отбор применён только к первому ряду.

Ячейка 9. Парный тест по невыровненным рядам: намеренная ошибка

```
1 stats.ttest_rel(first, last)  # парный t-тест
```

– `stats.ttest_rel(first, last)` — парный t-тест: берёт разности по позициям. Ряды разной длины, 16 и 29, и scipy останавливается: `ValueError: Array shapes are incompatible for broadcasting`. Ошибка намеренная.

Ячейка 10. Таблица «район на год»

```
by_year = pd.crosstab(dtp['district'], year)  # ДТП: район × год
by_year[[2015, 2025]].nlargest(4, 2015)  # четыре самых больших
```

– `pd.crosstab(dtp['district'], year)` — таблица: строки — районы, столбцы — годы, в клетках число ДТП. Пары выровнены по району сами; у района без ДТП в каком-то году — ноль.

– `by_year[[2015, 2025]]` — два столбца; их имена — числа, годы, поэтому пишутся без кавычек.

– `.nlargest(4, 2015)` — четыре строки с наибольшими значениями столбца 2015.

Ячейка 11. Парный тест, непарный и Вилкоксон для пар

```
by_year = by_year.drop('Брянская область')  # районы без Брянска
pairs = by_year.loc[by_year[2015] >= 20, [2015, 2025]]  # районы от 20 ДТП
paired = stats.ttest_rel(pairs[2015], pairs[2025])  # парный t-тест
unpaired = stats.ttest_ind(pairs[2015], pairs[2025], equal_var=False)  # ↩
↪ пары забыты
signed = stats.wilcoxon(pairs[2015], pairs[2025])  # Вилкоксон для пар
print(len(by_year), len(pairs), pairs.mean().round(1).tolist())
print((pairs[2015] - pairs[2025]).agg(['min', 'median', 'max']).tolist())  ↩
↪ # снижение по районам
print(paired.pvalue, unpaired.pvalue, signed.pvalue)
```

– `by_year.drop('Брянская область')` — таблица без строки с этой меткой: город Брянск убран.

– `by_year.loc[by_year[2015] >= 20, [2015, 2025]]` — районы от 20 ДТП в 2015 году и два столбца: пары готовы.

– `stats.ttest_rel(pairs[2015], pairs[2025])` — парный t-тест: разности внутри пар.

– `stats.ttest_ind(..., equal_var=False)` — тот же расчёт, как если бы годы были разными районами: пары забыты.

– `stats.wilcoxon(pairs[2015], pairs[2025])` — тест Вилкоксона для пар: знаковый ранговый тест по разностям.

– `pairs.mean()` — среднее каждого столбца, то есть каждого года.

– `(pairs[2015] - pairs[2025]).agg(['min', 'median', 'max'])` — снижение по районам: наименьшее, медиана, наибольшее.

4 Больше двух групп

Ячейка 12. ANOVA Уэлча, Крускал–Уоллис и попарные сравнения

```
1 unhurt = drivers.loc[drivers['health'] == 'Не пострадал', ↩
  ↪ 'experience_clean'].dropna()  # стаж не пострадавших
2 print(stats.f_oneway(dead, injured, unhurt, equal_var=False))  # ANOVA Уэлча
3 print(stats.kruskal(dead, injured, unhurt))  # Крускал-Уоллис
4 for name, a, b in [('погибшие и раненые', dead, injured), ('погибшие и не ↩
  ↪ пострадавшие', dead, unhurt),
5                    ('раненые и не пострадавшие', injured, unhurt)]:
6     p = stats.ttest_ind(a, b, equal_var=False).pvalue  # попарный t Уэлча
7     print(name, p, 'отвергаем' if p < 0.05 / 3 else 'не отвергаем')
```

– `stats.f_oneway(dead, injured, unhurt, equal_var=False)` — дисперсионный анализ трёх групп сразу; `equal_var=False` — ANOVA Уэлча. Результат с полями `statistic`, это F, и `pvalue`.

– `stats.kruskal(...)` — критерий Крускала–Уоллиса, двойник ANOVA по рангам.

– `for name, a, b in [...]:` — цикл по списку из трёх троек: название пары и две группы. Список записан на двух строках: Python ждёт продолжения, пока скобка не закрыта.

– Строка 6 — попарный t-тест Уэлча; строка 7 — решение с поправкой Бонферрони: порог `0.05 / 3`.

5 Две категории: хи-квадрат

Ячейка 13. Таблица сопряжённости

```
1 dark = dtp['light'].str.contains('темное').rename('тёмное время')  # ↩
  ↪ тёмное время суток
2 fatal = (dtp['dead_count'] > 0).rename('с погибшими')  # ДТП с погибшими
3 table = pd.crosstab(dark, fatal)  # таблица сопряжённости
4 table
```

– `dtp['light'].str.contains('темное')` — маска тёмного времени: значение содержит слово «темное», в данных без буквы «ё». `.rename('тёмное время')` — имя Series, оно станет подписью строк таблицы.

– `(dtp['dead_count'] > 0).rename('с погибшими')` — маска ДТП с погибшими с подписью столбцов.

– `pd.crosstab(dark, fatal)` — таблица 2 на 2: сколько ДТП в каждом сочетании.

Ячейка 14. Хи-квадрат независимости

```
1 chi = stats.chi2_contingency(table)  # хи-квадрат независимости
2 print(round(chi.statistic, 1), chi.pvalue, chi.dof)
3 chi.expected_freq.round(1)  # ожидаемые при независимости
```

– `stats.chi2_contingency(table)` — хи-квадрат независимости по таблице счётчиков. Результат с полями `statistic`, `pvalue`, `dof` — степени свободы — и `expected_freq` — таблица ожидаемых при независимости. У таблицы 2 на 2 scipy по умолчанию вносит поправку Йейтса.

– `chi.expected_freq.round(1)` — ожидаемые, округлённые до десятых; это массив NumPy, поэтому без подписей.

Ячейка 15. Коэффициент V Крамера и доли по строкам

```
plain = stats.chi2_contingency(table, correction=False)  # χ² без поправки Йейтса
v = np.sqrt(plain.statistic / (table.to_numpy().sum() * (min(table.shape) - 1)))  # V Крамера
print(round(v, 3))
shares = pd.crosstab(dark, fatal, normalize='index')  # доли по строкам
shares.round(3)
```

– `correction=False` — χ^2 без поправки Йейтса: по нему считают V.

– `table.to_numpy().sum()` — сумма всех клеток, число наблюдений n. `min(table.shape) - 1` — меньшее из чисел строк и столбцов минус один.

– Строка 2 — V Крамера по формуле: корень из χ^2, делённого на n и на это число.

– `pd.crosstab(dark, fatal, normalize='index')` — доли внутри каждой строки: какая доля ДТП с погибшими в светлое и в тёмное время.

Ячейка 16. Доля ДТП с погибшими по пяти условиям освещения

```
by_light = pd.crosstab(dtp['light'], fatal, normalize='index')[True].sort_values()  # доля ДТП с погибшими
fig, ax = plt.subplots(figsize=(8, 4))
by_light.plot.barh(ax=ax)  # столбцы по освещению
ax.set_title('Доля ДТП с погибшими по освещению')
ax.set_xlabel('Доля ДТП с погибшими')
ax.set_ylabel('')
by_light.round(3)
```

– `pd.crosstab(dtp['light'], fatal, normalize='index')[True]` — доли по строкам для всех пяти значений освещения; `[True]` — столбец «с погибшими».

– `.sort_values()` — по возрастанию доли; `by_light.plot.barh(ax=ax)` — горизонтальные столбцы, как на семинаре 8.

Ячейка 17. Доли вместо счётчиков: ловушка без ошибки

```
stats.chi2_contingency(shares)  # доли вместо счётчиков
```

– `stats.chi2_contingency(shares)` — в тест передана таблица долей из ячейки 15. Числа меньше единицы, поправка Йейтса обнуляет статистику: χ^2 0, p 1, без ошибки.

Ячейка 18. Отношение шансов

```
odds = (table.loc[True, True] / table.loc[True, False]) / (table.loc[False, True] / table.loc[False, False])  # отношение шансов
print(round(odds, 2), round(stats.fisher_exact(table).statistic, 2))
```

– `table.loc[True, True]` — клетка таблицы по меткам строки и столбца: тёмное время и ДТП с погибшими.

– Строка 1 — шанс гибели в темноте, делённый на шанс гибели днём; шанс — «с погибшими» к «без погибших».

– `stats.fisher_exact(table).statistic` — то же отношение шансов из теста Фишера.

Ячейка 19. Погода и тяжесть ДТП

```
four = dtp[dtp['weather'].isin(['Ясно', 'Пасмурно', 'Дождь', 'Снегопад']) & (dtp['severity'] != 'Без пострадавших (нет данных)')]  # частые погоды
weather = pd.crosstab(four['weather'], four['severity'])  # погода × тяжесть
chi_w = stats.chi2_contingency(weather)  # хи-квадрат независимости
v_w = np.sqrt(chi_w.statistic / (weather.to_numpy().sum() * (min(weather.shape) - 1)))  # V Крамера
print(round(chi_w.statistic, 1), round(chi_w.pvalue, 2), chi_w.dof, round(v_w, 3), round(chi_w.expected_freq.min(), 1))
```

– Строка 1 — ДТП четырёх частых погод без записи «Без пострадавших (нет данных)»: две маски через `&`, вторая в скобках, потому что это сравнение.

– `pd.crosstab(four['weather'], four['severity'])` — таблица 4 на 3.

– Строки 3–4 — хи-квадрат и V; у таблицы больше чем 2 на 2 поправки Йейтса нет, поэтому V считается по той же статистике.

– `chi_w.expected_freq.min()` — наименьшее ожидаемое: проверка правила «не меньше пяти».

Ячейка 20. Малая таблица: тест Фишера

```
small = dtp[dtp['district'] == 'Злынковский район']  # ДТП одного района
small_table = pd.crosstab(small['light'].str.contains('темное'), small['dead_count'] > 0)  # таблица 2 × 2
small_chi = stats.chi2_contingency(small_table)  # хи-квадрат на малой таблице
print(small_table.to_numpy().tolist(), round(small_chi.expected_freq.min(), 2))
print(round(stats.fisher_exact(small_table).pvalue, 2), round(small_chi.pvalue, 2))  # Фишер и хи-квадрат
```

– `pd.crosstab(small['light'].str.contains('темное'), small['dead_count'] > 0)` — таблица 2 на 2 по одному району; строки и столбцы заданы прямо масками.

– `small_table.to_numpy().tolist()` — четыре числа таблицы списком.

– `stats.fisher_exact(small_table).pvalue` — точный тест Фишера для таблицы 2 на 2; рядом — p хи-квадрата.

6 Значимость корреляции

Ячейка 21. Корреляция Пирсона и Спирмена с тестом

```
pair = drivers[['experience_clean', 'vehicle_age']].dropna()  # водители с обоими значениями
pearson = stats.pearsonr(pair['experience_clean'], pair['vehicle_age'])  # корреляция Пирсона
spearman = stats.spearmanr(pair['experience_clean'], pair['vehicle_age'])  # корреляция Спирмена
print(len(pair), round(pearson.statistic, 3), pearson.pvalue)
print(round(spearman.statistic, 3), spearman.pvalue)
```

– `drivers[['experience_clean', 'vehicle_age']].dropna()` — водители, у которых известны оба значения: пропуски убраны по двум столбцам сразу.

– `stats.pearsonr(x, y)` — коэффициент Пирсона и p; `stats.spearmanr(x, y)` — коэффициент Спирмена по рангам и p. Результаты с полями `statistic` и `pvalue`.

Ячейка 22. Корреляция по столбцам с пропусками: ловушка без ошибки

```
stats.pearsonr(drivers['experience_clean'], drivers['vehicle_age'])  # столбцы с пропусками
```

– `stats.pearsonr(...)` по столбцам таблицы с пропусками: `nan` вместо чисел, без ошибки.

7 Честный вывод и этап 3

Ячейка 23. Два новых вопроса для таблицы гипотез

```
exp_women = drivers.loc[drivers['gender'] == 'Женский', 'experience_clean'].dropna()  # стаж женщин за рулём
exp_men = drivers.loc[drivers['gender'] == 'Мужской', 'experience_clean'].dropna()  # стаж мужчин за рулём
gender_test = stats.ttest_ind(exp_women, exp_men, equal_var=False)  # t-тест Уэлча
d_gender = (exp_women.mean() - exp_men.mean()) / np.sqrt((exp_women.var() + exp_men.var()) / 2)  # d Коэна
defects = (dtp['road_defects'] > 0).rename('недостатки дороги')  # недостатки дорожных условий
defect_table = pd.crosstab(defects, fatal)  # таблица сопряжённости
chi_d = stats.chi2_contingency(defect_table)  # хи-квадрат независимости
plain_d = stats.chi2_contingency(defect_table, correction=False)  # χ² без поправки Йейтса
v_d = np.sqrt(plain_d.statistic / (defect_table.to_numpy().sum() * (min(defect_table.shape) - 1)))  # V Крамера
print(round(exp_women.mean(), 1), round(exp_men.mean(), 1), round(d_gender, 3))
print(round(chi_d.statistic, 2), round(chi_d.pvalue, 3), round(v_d, 3))
```

– Строки 1–4 — стаж женщин и мужчин за рулём: t-тест Уэлча и коэффициент d Коэна, как в ячейке 7.

– `(dtp['road_defects'] > 0).rename('недостатки дороги')` — маска ДТП с недостатками дорожных условий.

– Строки 6–9 — таблица сопряжённости, хи-квадрат с поправкой для p, без поправки для V, и коэффициент V Крамера.

Ячейка 24. Сводная таблица шести гипотез

```
report = pd.DataFrame([
    {'вопрос': 'стаж погибших и раненых водителей', 'тест': 't Уэлча', 'n': len(dead) + len(injured),
     'статистика': welch.statistic, 'p': welch.pvalue, 'эффект': d, 'размер': 'малый'},
    {'вопрос': 'ДТП в районах в 2015 и 2025 годах', 'тест': 'парный t', 'n': len(pairs),
     'статистика': paired.statistic, 'p': paired.pvalue, 'эффект': np.nan, 'размер': '-'},
    {'вопрос': 'тёмное время и ДТП с погибшими', 'тест': 'хи-квадрат', 'n': table.to_numpy().sum(),
     'статистика': chi.statistic, 'p': chi.pvalue, 'эффект': v, 'размер': 'малый'},
    {'вопрос': 'стаж и возраст машины', 'тест': 'Пирсон', 'n': len(pair),
     'статистика': pearson.statistic, 'p': pearson.pvalue, 'эффект': pearson.statistic, 'размер': 'ничтожный'},
    {'вопрос': 'стаж женщин и мужчин за рулём', 'тест': 't Уэлча', 'n': len(exp_women) + len(exp_men),
     'статистика': gender_test.statistic, 'p': gender_test.pvalue, 'эффект': d_gender, 'размер': 'большой'},
    {'вопрос': 'недостатки дороги и ДТП с погибшими', 'тест': 'хи-квадрат', 'n': defect_table.to_numpy().sum(),
     'статистика': chi_d.statistic, 'p': chi_d.pvalue, 'эффект': v_d, 'размер': 'ничтожный'},
], index=range(1, 7))
report['решение'] = np.where(report['p'] < 0.05, 'отвергаем', 'не отвергаем')  # решение при 0,05
report['после поправки'] = np.where(report['p'] < 0.05 / len(report), 'отвергаем', 'не отвергаем')  # порог 0,05 / 6
report.round(3)
```

– `pd.DataFrame([{...}, ...])` — таблица из списка словарей, как на семинаре 12; каждый словарь записан на двух строках.

– `'эффект': np.nan` и прочерк в `'размер'` — у парного t размера эффекта нет.

– `index=range(1, 7)` — строки пронумерованы с единицы.

– Последние строки — решение при 0.05 и после поправки Бонферрони, порог `0.05 / len(report)`.